

13b — Komplexität & Rekursion

Big-O als Sprache – und Funktionen, die sich selbst aufrufen

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Zählen statt Stoppen – Aufwand maschinen-unabhängig messen
2. Big-O und Landau-Notation – die Sprache der Komplexität
3. Die sechs Klassen: von $O(1)$ bis $O(2^n)$
4. Speicherkomplexität – der zweite Aufwand
5. Rekursion – Funktionen, die sich selbst aufrufen
6. **Klassiker: Fakultät & Fibonacci** – live programmiert

Lernziel: die Komplexitätsklasse einer Funktion am Code ablesen, die Landau-Definition in Worte übersetzen – und rekursive Funktionen schreiben und Schritt für Schritt nachvollziehen.

Heute: nach jedem Konzept *Live-Demo* (ich) und *Sofort ausprobieren* (Sie, Notebook 13b).

Motivation: zwei Wege, ein Ergebnis

```
def summe_a(n):                                # Schleife (Bestellnummern 1..n)
    summe = 0
    for i in range(1, n + 1):
        summe += i
    return summe
```

```
def summe_b(n):                                # Formel (13a!)
    return n * (n + 1) // 2
```

n	summe_a	summe_b
100	100 Schritte	1 Schritt
1 000 000	1 000 000 Schritte	1 Schritt

Unter der Haube: Zählen statt Stoppen

Warum nicht einfach die Stoppuhr nehmen?

summe_a(1 000 000)	Laptop (2 GHz)	Server (4 GHz)
gemessene Zeit	0.08 s	0.04 s
gezählte Operationen	1 000 000	1 000 000

- Die Stoppuhr misst **Rechner und Algorithmus gemischt**:
Taktfrequenz, Auslastung, Zufall (E4: jede CPU taktet anders)
- **Operationen zählen** ist maschinen-unabhängig – gleiche Zahl auf jedem Rechner, heute und in zehn Jahren

Die Informatik zählt **Schritte als Funktion von n** , nicht Sekunden – so wird der Algorithmus bewertet, nicht die Hardware.

Big-O - die Idee

- **O(...)** beschreibt das **Wachstumsverhalten** der Schrittzahl, nicht die genaue Zeit
- Konstanten und kleinere Terme fallen weg: $3n^2 + 5n$ ist **O(n²)**
- Wir betrachten in der Regel den **Worst Case** (schlechtester Fall)

Lesart

“Diese Funktion ist O(n).” = “Doppelt so große Eingabe → etwa doppelt so viele Schritte.”

Unter der Haube: die Landau-Notation, formal

$$f \in O(g) \iff \exists c > 0, n_0 : f(n) \leq c \cdot g(n) \text{ für alle } n \geq n_0$$

Dieselbe Aussage in Worten:

- $f(n) \leq c \cdot g(n)$: f wächst **höchstens** wie g – bis auf einen konstanten Faktor c
- “für alle $n \geq n_0$ ”: das muss erst **ab irgendwann** gelten – kleine n sind egal
- Big-O ist also eine **obere Schranke** für das Wachstum, keine exakte Laufzeit

Randnotiz: Ω bezeichnet die untere Schranke, Θ beides zugleich – in dieser Vorlesung genügt uns O .

Unter der Haube: warum Konstanten wegfallen

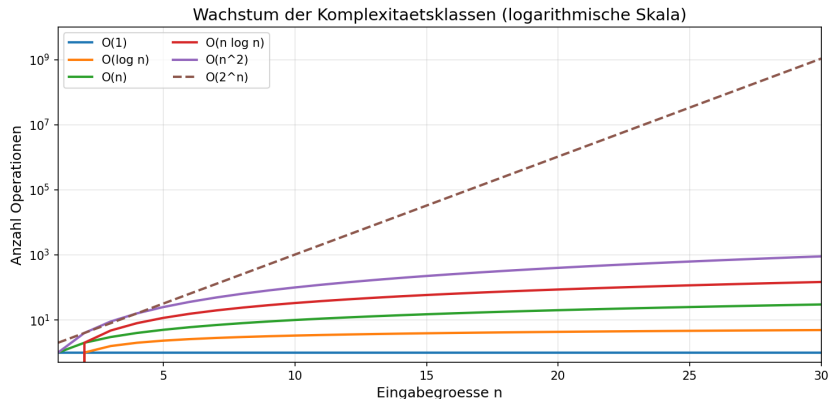
$3n^2 + 5n$ Operationen – welcher Term dominiert?

n	$3n^2$	$5n$	Anteil von $5n$
10	300	50	14 %
100	30 000	500	1.6 %
1 000	3 000 000	5 000	0.17 %

- Der kleinere Term wird bedeutungslos – Big-O behält nur den **dominanten Term**
- Die Definition erlaubt das formal: $3n^2 + 5n \leq 4n^2$ für alle $n \geq 5$, also gilt mit $c = 4$, $n_0 = 5$: $3n^2 + 5n \in O(n^2)$

Konstante Faktoren stecken im Rechner (Takt, Interpreter) – das **Wachstum** ist die Eigenschaft des Algorithmus.

Die sechs Klassen - die Wachstumskurven



Logarithmische Skala: jede Rasterlinie ist ein **Faktor 10**. $O(2^n)$ sprengt trotzdem jede Skala - schon bei $n = 30$ über eine Milliarde Schritte.

Die sechs Klassen - der Steckbrief

Klasse	Name	Typisches Beispiel
$O(1)$	konstant	Listenzugriff per Index
$O(\log n)$	logarithmisch	Halbieren pro Schritt: Binärsuche (14b)
$O(n)$	linear	Liste einmal durchlaufen
$O(n \log n)$	linearithmisch	gute Sortierer, <code>sorted()</code> (14a)
$O(n^2)$	quadratisch	doppelte Schleife über dieselbe Liste
$O(2^n)$	exponentiell	naive rekursive Fibonacci (heute!)

Merkhilfe $O(\log n)$: wer das Problem pro Schritt **halbiert**, braucht nur $\lceil \log_2 n \rceil$ Schritte – Telefonbuch in der Mitte aufschlagen. Vertiefung in 14b.

O(1) und O(n) am Code

```
def erste_bestellung(lager):    # O(1)
    return lager[0]
```

```
def gesamturnsatz(lager):      # O(n)
    summe = 0.0
    for preis in lager:
        summe += preis
    return summe
```

- erste_bestellung: **ein** Indexzugriff – Lagergröße egal
- gesamturnsatz: 1000-mal größeres Lager → 1000-mal mehr Schritte

► **Live-Demo** – 01_konstant_vs_linear.py (Operationen zählen)

$O(n^2)$ - verschachtelte Schleifen

```
def hat_duplikat(liste):  
    for i in range(len(liste)):  
        for j in range(i + 1, len(liste)):  
            if liste[i] == liste[j]:  
                return True  
    return False
```

- Außen n Durchläufe, innen im Mittel $n/2$: $n \cdot \frac{n}{2} = \frac{n^2}{2}$ Vergleiche $\rightarrow$ die Konstante $\frac{1}{2}$ fällt weg: **$O(n^2)$**

► **Live-Demo** - 02_komplexitaetstabelle.py, 03_ascii_balken.py

✎ **Sofort ausprobieren** - Notebook 13b, Übungen 3.1, 3.2 und 4.1

Speicherkomplexität – der zweite Aufwand

Was legt der Algorithmus neu an?	Speicher
einzelne Variablen (summe, i)	$O(1)$
neue Liste mit n Elementen	$O(n)$
Matrix mit $n \times n$ Einträgen	$O(n^2)$

- Es zählt nur der **zusätzliche** Speicher – die Eingabe selbst nicht
- gesamtumsatz von eben: Zeit $O(n)$, Speicher **$O(1)$** – nur eine Akku-Variable
- Zeit und Speicher werden **getrennt** analysiert, beide in Big-O

 **Sofort ausprobieren** – Notebook 13b, Übungen 5.1/5.2: Speicherkomplexität analysieren

Rekursion - eine Funktion ruft sich selbst

```
def fakultaet(n):  
    if n == 0:                                # 1) Basisfall  
        return 1  
    return n * fakultaet(n - 1) # 2) rekursiver Fall
```

- **Basisfall:** der kleinste Fall, Antwort steht direkt da – **ohne** Selbstaufruf
- **Rekursiver Fall:** führt das Problem auf ein **strikt kleineres** zurück
- Jede Rekursion braucht **beide** Teile – den Basisfall zuerst schreiben!

Mathematische Parallele: $0! = 1$ und $n! = n \cdot (n - 1)!$ – der Code **ist** die mathematische Definition, Zeile für Zeile.

Unter der Haube: fakultaet(4) auf dem Call-Stack

Phase	offene Aufrufe (Stack)	Ergebnis
Abstieg	f(4)	wartet auf f(3)
...	f(4) f(3) f(2) f(1) f(0)	Basisfall: 1
Aufstieg	f(4) f(3) f(2) f(1)	$1 * 1 = 1$
...	f(4) f(3) f(2)	$2 * 1 = 2$
...	f(4) f(3)	$3 * 2 = 6$
...	f(4)	$4 * 6 = 24$

FS 07 (“was ein Aufruf wirklich tut”) gilt unverändert: jeder rekursive Aufruf ist ein **ganz normaler Aufruf** mit **eigenen lokalen Namen** – jedes f hat sein eigenes n, alle liegen gleichzeitig auf dem Stack.

Unter der Haube: der Stack kostet Speicher

- Iterativ (Schleife): eine Akku-Variable wird überschrieben → Speicher **$O(1)$**
- Rekursiv: am tiefsten Punkt sind **alle n Aufrufe gleichzeitig offen**
– jeder Frame belegt Platz → Speicher **$O(n)$**
- **Fehlender Basisfall** (oder n schrumpft nicht): der Stapel wächst endlos – Python zieht bei ca. 1000 offenen Aufrufen die Notbremse: **RecursionError**
- Das Terminierungs-Argument aus FS 10 gilt wörtlich: der Parameter muss pro Aufruf **strikt Richtung Basisfall schrumpfen**

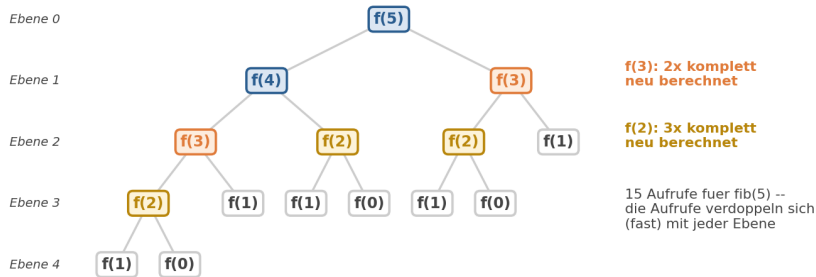
► **Live-Demo** – 04_speicher_iterativ_vs_rekursiv.py: die Treppe der offenen Aufrufe

Fibonacci - Rekursion mit zwei Selbstaufrufen

```
def fib_rekursiv(n):  
    if n < 2:           # Basisfaelle: fib(0)=0, fib(1)=1  
        return n  
    return fib_rekursiv(n-1) + fib_rekursiv(n-2)
```

- Die Folge 0, 1, 1, 2, 3, 5, 8, 13, ... – jedes Glied ist die **Summe der zwei Vorgänger**
- Wieder gilt: der Code ist die Definition $F_n = F_{n-1} + F_{n-2}$
- Elegant – aber **zwei** Selbstaufrufe pro Ebene haben einen Preis ...

Unter der Haube: der Rekursionsbaum von fib(5)



`fib(3)` wird **2-mal**, `fib(2)` **3-mal** komplett neu berechnet – die Aufrufe verdoppeln sich (fast) mit jeder Ebene: das ist **$O(2^n)$** , die oberste Kurve der Klassen-Grafik.

Fibonacci iterativ - dieselbe Folge in $O(n)$

```
def fib_iterativ(n):  
    a, b = 0, 1          # zwei Merkvariablen  
    for i in range(n):  
        a, b = b, a + b  # Schleifen-Muster (FS 10)  
    return a
```

n	rekursiv: Aufrufe	iterativ: Durchläufe
10	177	10
20	21 891	20
30	2 692 537	30

Gleiches Ergebnis, radikal anderes Wachstum: **$O(2^n)$ vs. $O(n)$** . Im Klassiker gleich **spüren** Sie den Unterschied.

Klassiker: Fakultät & Fibonacci - die Aufgabe

Der Rekursions-Klassiker der Programmierschule:

- Schreibe `fakultaet(n)` **rekursiv** - Basisfall $0! = 1$ zuerst
- Schreibe `fib_rekursiv(n)` und `fib_iterativ(n)`
- Gib $4!$ aus und vergleiche beide Fibonacci-Varianten für $n = 10$ und $n = 30$
- Bei $n = 30$ den Unterschied **spüren**: kurz warten vs. sofort

So soll es laufen

$4! = 24$

`fib(10) rekursiv:` 55

`fib(10) iterativ:` 55

`fib(30) rekursiv:` 832040 (spuerbar langsam)

`fib(30) iterativ:` 832040 (sofort)

Klassiker: Fakultät & Fibonacci - Live-Coding

► **Live-Coding ab leerem Editor** - Referenz: `90_klassiker_fakultaet_fibonacci.py`

Teilziele:

1. `fakultaet(n)` - **Basisfall zuerst**, dann rekursiver Fall
2. Trace von `fakultaet(4)` an der Konsole nachvollziehen
3. `fib_rekursiv(n)` mit zwei Basisfällen ($n < 2$)
4. `fib_iterativ(n)` daneben stellen (Schleifen-Muster aus FS 10)
5. Beide für $n = 30$ laufen lassen - die Dauer mit $O(2^n)$ vs. $O(n)$ erklären

📎 **Zum Selberlösen** - Handout "*Klassiker 13b: Fakultät & Fibonacci*" (Klassiker/13b_Klassiker_Fakultaet_Fibonacci.pdf). Vertiefung: Notebook 13b, Kapitel Rekursion (Bonus: Türme von Hanoi).

Cheat Card

Aufgabe	Regel / Syntax
eine Schleife über n	$O(n)$
verschachtelte Schleifen	$O(n^2)$
Problem pro Schritt halbieren	$O(\log n)$
Landau formal	$f(n) \leq c \cdot g(n)$ ab n_0
Basisfall	if $n == 0$: return 1
rekursiver Fall	return $n * \text{fakultaet}(n - 1)$
Speicher rekursiv	$O(n)$ – ein Frame pro offenem Aufruf
Klassiker heute	Fakultät rekursiv; fib: $O(2^n)$ vs. $O(n)$

Faustregel: Schleifen zählen, dominanten Term behalten. Bei Rekursion: *Basisfall* zuerst – und schrumpft der Parameter strikt?

Ausblick: 14a - Sortieralgorithmen

Unser `fib_rekursiv` war **korrekt, aber unbrauchbar langsam** – die Wahl des Algorithmus entscheidet. Nächste Woche dieselbe Frage beim **Sortieren**:

```
# Vorschau 14a: Bubble Sort --  $O(n^2)$ 
```

```
for i in range(n):  
    for j in range(n - i - 1):  
        if liste[j] > liste[j + 1]:  
            liste[j], liste[j+1] = liste[j+1], liste[j]
```

- Bubble/Insertion Sort **$O(n^2)$** vs. `sorted()` **$O(n \log n)$** – heute gelernt, dort gespürt
- ASCII-Balken (Demo 3) und **Rekursion** kehren zurück: Quicksort sortiert rekursiv

Heute geübt

- ✓ Zählen statt Stoppen: Operationen als maschinen-unabhängiges Maß
- ✓ Big-O als obere Schranke; Landau-Definition in Worte übersetzt
- ✓ Warum Konstanten und kleinere Terme wegfallen (c, n_0)
- ✓ Die sechs Klassen von $O(1)$ bis $O(2^n)$ – am Code abgelesen
- ✓ Speicherkomplexität getrennt von der Zeit analysiert
- ✓ **Rekursion**: Basisfall, rekursiver Fall, Call-Stack, RecursionError
- ✓ **Klassiker Fakultät & Fibonacci** live gesehen – $O(2^n)$ gespürt

Ihre Aufgaben bis zur nächsten Woche:

- Klassiker **Fakultät & Fibonacci** eigenständig lösen (Handout)
- “Sofort ausprobieren“-Übungen 3.1–5.2 im Notebook 13b
- Abschlussübungen 1–4: `finde_minimum`, `enthält_element`, `zähle_vorkommen`, `matrix_transponieren`